

LAB 7

Question 1:

Code

```
#include<iostream>
using namespace std;
class Node{
public:
    int id;
    string name;
    float price;
    Node *next;
    Node(int ID, string n, float p) : id(ID),name(n), price(p), next(nullptr) {}
};
class Queue{
private:
    Node *front, *rear;
public:
    Queue() : front(nullptr), rear(nullptr) {}

    bool isEmpty() {
        return front == nullptr;
    }

    void enqueue(int id, string name, float price) {
        Node *newNode = new Node(id, name, price);
        if(rear == nullptr) front = rear = newNode;
        else {
            rear->next = newNode;
            rear = newNode;
        }
        newNode->next = front;
    }

    void dequeue() {
        if(front == nullptr) {
            cout<<"No order is in process"<<endl;
            return;
        }
        if(rear == front) {
            delete front;
            front = rear = nullptr;
        } else {
            Node *temp = front;
            front = front->next;
            rear->next = front;
            delete temp;
        }
    }
}
```

```

void availableItems() {
    cout<<"ID \t Item \t\t Price"<<endl;
    cout<<"1 \t Zinger Burger \t 500.25"<<endl;
    cout<<"2 \t French Fries \t 250.00"<<endl;
    cout<<"3 \t Chiken Roll \t 250.00"<<endl;
    cout<<"4 \t Pizza \t\t 1250.00"<<endl;
    cout<<"5 \t Sandwich \t 375.00"<<endl;
    cout<<"6 \t Cold Drink \t 250.00"<<endl;
    cout<<"7 \t Tea \t\t 70.00"<<endl;
    cout<<"8 \t Coffee \t 450.00"<<endl;
}

void placeOrder(int id) {
    switch(id) {
        case 1:
            enqueue(1, "Zinger Burger", 500.25);
            break;
        case 2:
            enqueue(2, "French Fries", 250.00);
            break;
        case 3:
            enqueue(3, "Chicken Roll", 350.00);
            break;
        case 4:
            enqueue(4, "Pizza", 1250.25);
            break;
        case 5:
            enqueue(5, "Sandwich", 375.00);
            break;
        case 6:
            enqueue(6, "Cold Drink", 350.00);
            break;
        case 7:
            enqueue(7, "Tea", 70.00);
            break;
        case 8:
            enqueue(8, "Coffee", 450.00);
            break;
        default:
            cout<<"Invalid item id"<<endl;
    }
}

void processNextOrder() {
    if(!isEmpty()) {
        cout<<"Processing order: "<< front->name<<" - $" << front->price<<endl;
        dequeue();
    }
}

void display() {

```

```

        if(front == nullptr) {
            cout<<"No order to display"<<endl;
            return;
        }
        Node *temp = front;
        cout<<"ID \t Item \t\t Price"<<endl;
        do {
            cout<<temp->id<<" \t"<<temp->name<<"\t"<<temp->price<<endl;
            temp = temp->next;
        } while (temp != front);
    }
};

int main() {
    Queue q;
    int choice, id;
    do {
        cout<<"Cafeteria Ordering System"<<endl;
        cout<<"1. Display Menu"<<endl;
        cout<<"2. Place Order"<<endl;
        cout<<"3. Process Next Order"<<endl;
        cout<<"4. Display Current Orders"<<endl;
        cout<<"5. Exit"<<endl;
        cout<<"Enter your choice"<<endl;
        cin >> choice;

        switch(choice) {
            case 1:
                q.availableItems();
                break;
            case 2:
                cout<<"Enter item id"<<endl;
                cin >> id;
                q.placeOrder(id);
                break;
            case 3:
                q.processNextOrder();
                break;
            case 4:
                q.display();
                break;
            default:
                cout<<"Invalid choice"<<endl;
        }
    } while (choice != 5);
    if(choice == 5) cout<<"Exiting..."<<endl;
    return 0;
}

```

Question 2:

Code

```
#include<iostream>
using namespace std;
class Queue{
private:
    string *patient;
    int size, front, rear, elements;
public:
    Queue(int s) {
        size = s;
        patient = new string[size];
        front = -1;
        rear = -1;
        elements = 0;
    }
    bool isFull() {
        return elements == size;
    }
    bool isEmpty() {
        return elements == 0;
    }

    void enqueue(string name) {
        if(isFull()) {
            cout<<"Waiting area is full"<<endl;
            return;
        }
        if(isEmpty()) {
            front = 0;
        }
        rear = (rear + 1) % size;
        patient[rear] = name;
        elements++;
    }

    void dequeue() {
        if(isEmpty()) {
            cout<<"No patient available"<<endl;
            return;
        }
        front = (front + 1) % size;
        elements--;
        if(elements == 0) {
            front = -1;
            rear = -1;
        }
    }

    string getNextPatient() {
```

```

        if(isEmpty()) {
            cout<<"No patient present"<<endl;
            return;
        }
        return patient[front];
    }

    void display() {
        if(isEmpty()) {
            cout<<"No patients available"<<endl;
            return;
        }
        for(int i = front; i == rear; i = (i + 1) % size) cout<<patient[i]<<" ";
        cout<<endl;
    }

    ~Queue() {
        delete[] patient;
    }
};

int main() {
    int seats;
    cout<<"Enter the seats available for patients"<<endl;
    cin >> seats;

    Queue q(seats);

    int choice;
    string patientName;

    do {
        cout << "\nDoctor's Office Queue Management" << endl;
        cout << "1. Add Patient to Queue" << endl;
        cout << "2. Send next patient to doctor" << endl;
        cout << "3. See who's next" << endl;
        cout << "4. Display patient queue" << endl;
        cout << "5. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;
        cin.ignore();

        switch (choice) {
            case 1:
                cout << "Enter patient name: ";
                getline(cin, patientName);
                q.enqueue(patientName);
                break;
            case 2:
                if (!q.isEmpty()) {
                    cout << "Sending " << q.getNextPatient() << " to the doctor." << endl;
                    q.dequeue();
                }
            }
        }
    } while (choice != 5);
}

```

```
        }  
        break;  
    case 3:  
        cout << "Next patient: " << q.getNextPatient() << endl;  
        break;  
    case 4:  
        q.display();  
        break;  
    case 5:  
        cout << "Exiting system..." << endl;  
        return 0;  
    default:  
        cout << "Invalid choice" << endl;  
    }  
} while (choice != 5);  
  
return 0;  
}
```